

# Image Processing with Deep Learning

Dr. Ei Moh Moh Aung  
FCS



# ImageNet

- ImageNet is a **large-scale image database** designed for visual object recognition software research. It contains over **14 million images** and **2200 categories**.
- ImageNet was created by the **Stanford Vision Lab** and first released in **2009**.
- It has since become a **benchmark for evaluating the performance of visual object recognition software**.
- ImageNet has been used to train a number of successful **deep learning models**, including **AlexNet**, **VGGNet**, **ResNet**, and **Inception**.

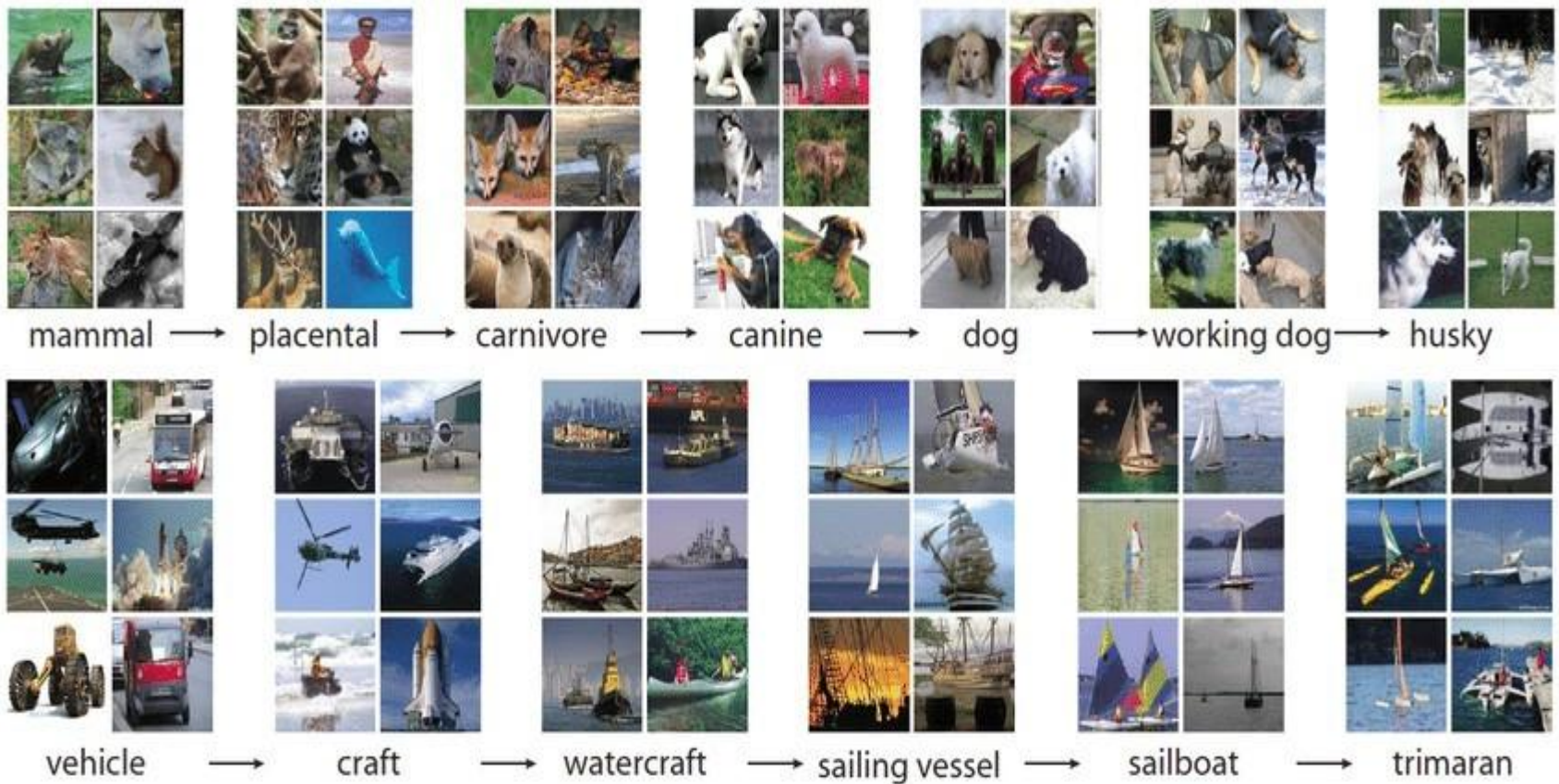
# ImageNet: Benefits

- It is a large and diverse dataset, which allows for the training of more acc
- The images are organized into a **hierarchical taxonomy**, which makes it easier to train models for specific tasks.
- The ImageNet Large Scale Visual Recognition Challenge (**ILSVRC**) provides a benchmark for evaluating the performance of visual object recognition software.

# ImageNet: Limitations

- The images are **all labeled by humans**, which can be a time-consuming and expensive process.
- The dataset is biased towards **Western culture**, which can make it less effective for training models for other cultures.
- The dataset is not always up-to-date, which can make it less effective for training models for new objects or scenes.

# ImageNet Dataset



# ImageNet Web link



IMGENET

14,197,122 images, 21841 synsets indexed

[Home](#) [Download](#) [Challenges](#) [About](#)

Not logged in. [Login](#) | [Signup](#)

**ImageNet** is an image database organized according to the **WordNet** hierarchy (currently only the nouns), in which each node of the hierarchy is depicted by hundreds and thousands of images. The project has been **instrumental** in advancing computer vision and deep learning research. The data is available for free to researchers for non-commercial use.

**Mar 11 2021. ImageNet website update.**

© 2020 Stanford Vision Lab, Stanford University, Princeton University [imagenet.help.desk@gmail.com](mailto:imagenet.help.desk@gmail.com) Copyright infringement

# AlexNet

- **What is AlexNet?**
  - A **Convolutional Neural Network (CNN)** architecture designed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton.
  - Winner of the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012. AlexNet achieved a major breakthrough on ImageNet.
  - It marked the resurgence of Deep Learning and proved that CNNs could **outperform** traditional hand-crafted feature extraction methods (like SIFT or HOG).

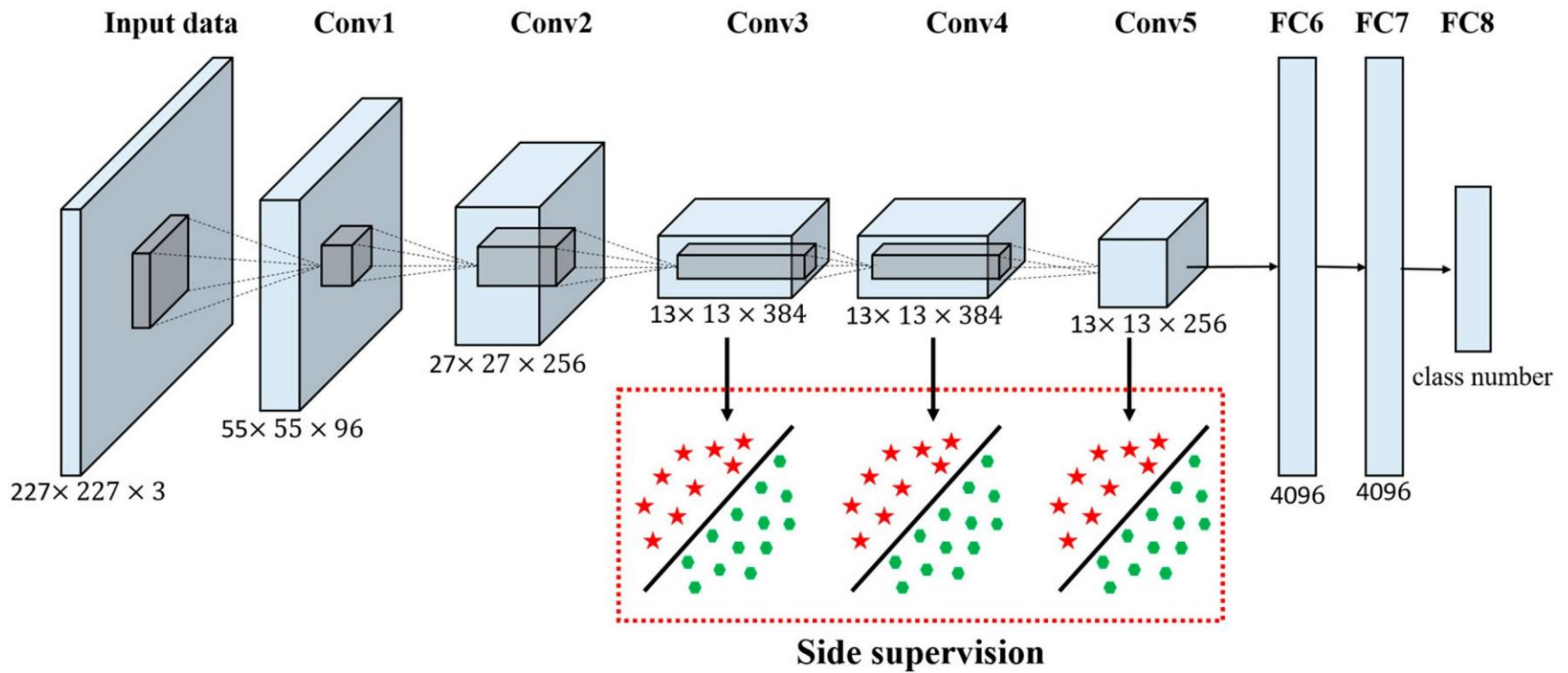
# The Problem & Context

- **The Dataset:**
  - ImageNet contains over 15 million labeled high-resolution images belonging to roughly 22,000 categories.
- **The Challenge (ILSVRC):**
  - A subset of ImageNet with 1,000 classes and 1.2 million training images.
- **The Goal:**
  - Reduce the Top-1 and Top-5 error rates in image classification.
- **Hardware Limitation:**
  - GPUs in 2012 had limited memory (GTX 580 had only 3GB), requiring innovative architecture splitting.

# Architecture Overview

- Depth: 8 Learned Layers.
  - 5 Convolutional Layers.
  - 3 Fully Connected (Dense) Layers.
- Parameters: Approximately 60 million parameters.  
Neurons: 650,000 neurons.
- Input: Accepts  $224 \times 224 \times 3$  pixel images (RGB channels).

# AlexNet



# AlexNet

| Layer      | # filters / neurons | Filter size | Stride | Padding | Size of feature map | Activation function |
|------------|---------------------|-------------|--------|---------|---------------------|---------------------|
| Input      | -                   | -           | -      | -       | 227 x 227 x 3       | -                   |
| Conv 1     | 96                  | 11 x 11     | 4      | -       | 55 x 55 x 96        | ReLU                |
| Max Pool 1 | -                   | 3 x 3       | 2      | -       | 27 x 27 x 96        | -                   |
| Conv 2     | 256                 | 5 x 5       | 1      | 2       | 27 x 27 x 256       | ReLU                |
| Max Pool 2 | -                   | 3 x 3       | 2      | -       | 13 x 13 x 256       | -                   |
| Conv 3     | 384                 | 3 x 3       | 1      | 1       | 13 x 13 x 384       | ReLU                |
| Conv 4     | 384                 | 3 x 3       | 1      | 1       | 13 x 13 x 384       | ReLU                |
| Conv 5     | 256                 | 3 x 3       | 1      | 1       | 13 x 13 x 256       | ReLU                |
| Max Pool 3 | -                   | 3 x 3       | 2      | -       | 6 x 6 x 256         | -                   |
| Dropout 1  | rate = 0.5          | -           | -      | -       | 6 x 6 x 256         | -                   |

# AlexNet: Conclusion

- AlexNet was a combination of efficient GPU usage, novel activation functions (ReLU), and aggressive regularization (Dropout/Augmentation).
- Used Rectified Linear Units (ReLU):  
formula:  $f(x) = \max(0, x)$
- It proved that “Deeper is Better” for image recognition.
- It remains one of the most influential papers in the history of Computer Science.

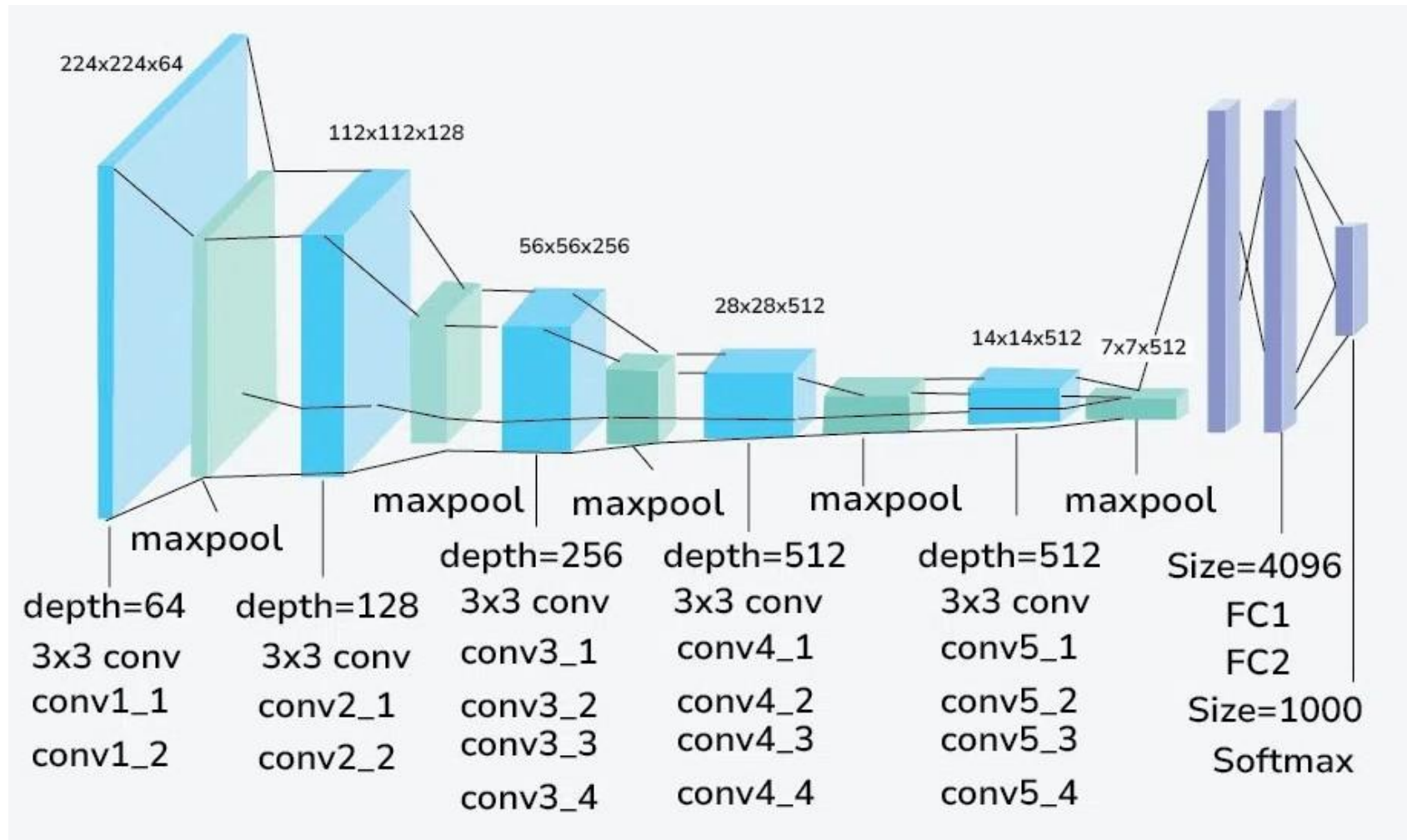
# VGGNet

- What is VGGNet?
  - A CNN architecture developed by the **Visual Geometry Group** (VGG) at Oxford University (Simonyan & Zisserman, **2014**).
- The Context:
  - Following AlexNet (2012), researchers asked: "Does depth really matter?"
- The Goal:
  - To investigate the **effect of increasing network depth** on accuracy **using very small convolution filters**.
- Instead of large filters (like 11x11 or 5x5 in AlexNet), **VGG uses only 3x3 filters**.
- Several small filters can gradually see a larger region of the original image.

# Key Architecture Design

- The "Small Filter" Philosophy:
  - Instead of large filters (like 11x11 or 5x5 in AlexNet), **VGG uses only 3x3 filters**.
  - Two 3x3 layers have the **same receptive field** as one 5x5 layer but **fewer parameters** and more non-linearity (ReLU).
- **Uniformity**: The architecture is extremely uniform.
  - Conv layers: 3x3 filters, stride 1, padding 1 (same padding).
  - Pooling layers: 2x2 max pooling, stride 2.
  - Depth: Pushed the **depth to 16-19 weight layers** (VGG-16 and VGG-19).

# Key Architecture Design



# VGG-16 vs. VGG-19 Configurations

- **VGG-16:** 13 Convolutional layers + 3 Fully Connected layers.
  - Blocks: 2 convs -> pool -> 2 convs -> pool -> 3 convs -> pool -> 3 convs -> pool -> 3 convs -> pool.
- **VGG-19:** 16 Convolutional layers + 3 Fully Connected layers.
  - Simply adds more conv layers to the deeper blocks.
- **Parameter Count:** Massive! Approx. **138 million parameters** (mostly in the first fully connected layers).
- A parameter is a learnable number in the network. For a convolution layer, parameters are mainly the **weights and biases**.

# Impact & Limitations

- Strengths:
  - State-of-the-art accuracy in 2014.
  - Excellent feature extractor for other tasks (Transfer Learning).
  - Simple, elegant, and easy to implement.
- Weaknesses:
  - **Slow to train**: Due to depth and number of parameters.
  - **Memory Heavy**: The massive dense layers require significant GPU memory.
  - **Inefficient**: Later architectures (ResNet, Inception) achieved better accuracy with far fewer parameters.

# ResNet (Residual Network)

- What is ResNet?
  - A Convolutional Neural Network architecture introduced by Microsoft Research (Kaiming He et al.) in 2015.
- The Context:
  - Following VGG (19 layers) and GoogleNet (22 layers), researchers wanted to go deeper.
- The Problem:
  - Simply stacking **more layers led to higher training error**, not just overfitting. This was the "degradation problem."
- Achievement:
  - Winner of ILSVRC 2015 with an astounding **depth of 152 layers**.

# The Degradation Problem

- **Counter-intuitive Finding:** With plain networks (like VGG), adding more layers eventually causes accuracy to saturate and then degrade rapidly.
- Why? It's harder to optimize very deep networks.
  - Vanishing/Exploding Gradients (mostly solved by normalization).
  - Ideally, extra layers should just be identity mappings if they aren't needed, but solvers struggle to learn this  $f(x)=x$  function.

# The Core Idea: Residual Learning

- Concept:
  - Instead of learning a direct mapping  $H(x)$ , let the network learn a residual mapping  $F(x)=H(x)-x$ .
- Reformulation:
  - The original mapping becomes
$$H(x)=F(x)+x.$$
- Intuition:
  - It is easier to optimize the residual (the "difference") to zero than to learn an **identity mapping** from scratch.
  - If identity is optimal, the weights are driven to 0.

# The Residual Block (Skip Connection)

- Structure:
  - Input  $x$  goes through weight layers (Conv-ReLU-Conv).
  - The original input  $x$  is added element-wise to the output of the weight layers.
  - Formula:  $y = F(x, W_i) + x$ .
- Skip Connection:
  - Also called a shortcut connection. It bypasses one or more layers.
- Benefit:
  - Allows gradients to flow directly through the network during backpropagation, mitigating the vanishing gradient problem.

# Architecture Variants (ResNet-18 to ResNet-152)

- **Basic Block:**
  - Used in ResNet-18 and ResNet-34.
  - Two  $3 \times 3$  convolutions.
- **Bottleneck Block:**
  - Used in ResNet-50, ResNet-101, and ResNet-152.
    - Three layers:  $1 \times 1$ ,  $3 \times 3$ ,  $1 \times 1$ .
    - The  $1 \times 1$  layers reduce and then restore dimensions, making deep networks computationally affordable.

# Bottleneck Design Details

- Motivation:
  - To increase depth without exploding parameters or computation time.
- Structure:
  - $1 \times 1$  Conv (Reduce channels, e.g., 256  $\rightarrow$  64).
  - $3 \times 3$  Conv (Process features, e.g., 64  $\rightarrow$  64).
  - $1 \times 1$  Conv (Restore/Expand channels, e.g., 64  $\rightarrow$  256).
- Result:
  - A deeper network (152 layers) that still has lower complexity than VGG-19.

# Implementation Details

- Input:
  - 224×224 images.
- Stem:
  - Starts with a 7×7 conv, stride 2, followed by max pooling.
- Stages:
  - 4 main stages of residual blocks with increasing filter counts (64, 128, 256, 512).
- Output:
  - Global Average Pooling layer -> 1000-way Fully Connected layer -> Softmax.
- No Dropout:
  - Batch Normalization is used after every convolution, acting as a regularizer.

# ResNet: Architectures

- Here are some of the most popular ResNet architectures:
  - ResNet50
  - ResNet101
  - ResNet152
  - ResNet200
  - ResNet34
  - ResNet18

# ResNet: Summary

- ResNet solved the **degradation problem** using simple identity skip connections.
- It demonstrated that depth is crucial for high-level feature extraction.
- The architecture is modular, efficient, and remains a primary backbone for many Computer Vision tasks today (Object Detection, Segmentation).

- 

END